

P3500R0: Standardizing `std::tensor` for Deep Learning and High-Performance Data-Intensive Computing in ISO C++

Executive Summary and Problem Statement

In the Python data science ecosystem, the `numpy.ndarray` object serves as the fundamental interchange format for numerical computing. Higher-level frameworks—including PyTorch, TensorFlow, JAX, SciPy, and OpenCV—build directly upon NumPy's unified concept of contiguous and strided multidimensional array buffers. This shared foundation enables seamless, zero-copy data exchange across heterogeneous software boundaries.

In contrast, the C++ software landscape features a fragmented ecosystem of incompatible multidimensional array and tensor implementations. High-performance software engineering in C++ relies on disconnected third-party abstractions, such as Eigen Tensor, LibTorch `at::Tensor`, TensorFlow Core Tensor, OpenCV `cv::Mat`, Armadillo, Blaze, and xtensor. Because these libraries do not share a common standard owning container type, integrating components across C++ library boundaries introduces significant software overhead¹.

Passing multidimensional array data between disparate C++ third-party libraries currently forces developers to choose between two sub-optimal architectures:

1. **Deep Memory Copies:** Allocating new memory buffers and copying element data across library boundaries introduces significant memory bandwidth penalties and execution latency¹. In deep learning pipelines where data-intensive tensor transformations execute continuously across host and accelerator devices, deep copying degrades operational throughput¹.
2. **Opaque Raw Pointer Casting:** Bypassing type safety by extracting raw data pointers (such as `void*` or `float*`) strips away crucial metadata, including dynamic shape extents, stride configurations, element data types, layout policies, and ownership semantics². This approach compromises compile-time type safety, increases the risk of memory leaks and lifetime errors, and breaks thread safety invariants².

While recent C++ standards have introduced essential non-owning multidimensional abstractions—such as `std::mdspan` in C++23⁵ and dense linear algebra algorithms in `std::linalg` for C++26⁶—the Standard Template Library (STL) still lacks a universal, owning, dynamic multidimensional container. Proposed owning adapters such as `std::mdarray` (P1684 / P3308) provide valuable container wrappers, but they enforce static rank constraints at compile time, lack native awareness of heterogeneous hardware memory spaces (such as CUDA, ROCm, and SYCL memory domains), and omit standard Application Binary Interface (ABI) protocols for zero-copy cross-language exchange².

This paper presents a formal design blueprint for `std::tensor`, a proposed addition to the ISO C++ Standard Template Library under header `<tensor>`. Designed as a multidimensional,

heterogeneous-aware container abstraction, `std::tensor` bridges compile-time layout optimizations with dynamic runtime rank flexibility. It integrates directly with existing C++ facilities while establishing a standard C ABI bridge based on the established DLPack protocol (DLManagedTensorVersioned) for zero-copy data exchange across C++ libraries, Python runtimes, and hardware accelerators².

Architectural Analysis of ISO C++ Multidimensional Primitives

Evaluating the design space for `std::tensor` requires examining the capabilities and structural limitations of existing and proposed ISO C++ multidimensional array facilities.



The Non-Owning Abstraction: `std::mdspan`

Standardized in C++23 via P0009, `std::mdspan` provides a non-owning multidimensional view over a contiguous or strided sequence of elements⁵. The architecture of `std::mdspan` decouples multidimensional indexing from element storage through four template

parameters⁵:

1. **Element Type (ElementType)**: Defines the object type stored in the underlying sequence⁵.
2. **Extents (Extents)**: Represents the multidimensional index space domain using a combination of compile-time static extents (`std::static_extent`) and runtime dynamic extents (`std::dynamic_extent`)⁵.
3. **Layout Policy (LayoutPolicy)**: Maps a multidimensional index tuple $(i_0, i_1, \dots, i_{k-1})$ to a 1D scalar memory offset⁵. Standard policies include `layout_left` (column-major), `layout_right` (row-major), `layout_stride` (arbitrary striding), and padded variations (`layout_left_padded`, `layout_right_padded` via P2642)⁵.
4. **Accessor Policy (AccessorPolicy)**: Governs element dereferencing semantics, enabling default pointer dereferencing, overaligned SIMD accessors (`aligned_accessor` via P2897), or specialized atomic accessors⁵.

Although `std::mdspan` serves as an efficient viewing abstraction, it does not manage storage memory⁵. It relies on external memory allocations whose lifetimes must exceed that of the `mdspan` instance.

Slicing and Views: `submdspan`

Proposal P2630 (`submdspan`) introduces subview extraction capabilities for `std::mdspan`¹³. By passing slice specifiers—such as full range markers (`full_extent`), scalar indices, index pairs, or strided range descriptors (`strided_slice`)—developers can generate sub-dimensional views without altering underlying element storage¹³.

Subsequent enhancements (P3355, P3663) extend `submdspan` to support user-defined pair-like slice types and preserve compile-time layout properties, preventing structured layouts (e.g., `layout_left_padded`) from needlessly devolving into generic `layout_stride` mappings¹¹.

Algorithmic Linear Algebra: `std::linalg`

Proposal P1673 introduces a set of free-function algorithms for matrix and vector arithmetic operating directly on `std::mdspan` views⁶. It standardizes classic BLAS Level 1, 2, and 3 operations—such as `matrix_vector_product` and `matrix_product`—within the standard library⁷. Because `std::linalg` functions accept `std::mdspan` parameters, they are storage-agnostic⁵. However, this non-owning design requires callers to separately allocate and manage output memory buffers, underscoring the need for a complementary owning container abstraction⁵.

Owning Container Adapters: `std::mdarray`

To provide an owning counterpart to `std::mdspan`, proposal P1684 (updated in P3308) introduces `std::mdarray`, an owning multidimensional container adapter⁸. `std::mdarray` wraps a 1D sequential container—such as `std::vector<T>` or `std::array<T, N>`—and mirrors the `Extents` and `LayoutPolicy` interfaces of `mdspan`⁸.

Despite its utility as a general container adapter, `std::mdarray` exhibits key structural limitations when applied to data-intensive deep learning workloads:

- **Static Rank Enforcement:** The rank N of `std::mdarray` is fixed at compile time via its Extents template argument⁸. Deep learning workloads frequently require dynamic runtime ranks, where tensor dimensionality varies across computational graph layers or input batches².
- **Container Adapter Overhead:** Delegate storage management to an underlying 1D container introduces ambiguities around moved-from object states, allocation alignment, and restrictive capacity models⁸.
- **Absence of Heterogeneous Memory Awareness:** `std::mdarray` does not account for execution domains across CPU host memory, CUDA device memory, ROCm host-pinned memory, or SYCL unified memory spaces².
- **Lack of Standard C ABI Interoperability:** `std::mdarray` lacks built-in support for zero-copy binary data exchange with external C APIs, Python runtimes, or C-based foreign function interfaces (FFIs)².

Multidimensional Abstraction	Ownership Model	Rank Determination	Memory Layout Support	Heterogeneous Memory Support	Standard C ABI Exchange Protocol
<code>std::mdspan</code> [cite: 5]	Non-owning View	Static Rank (Compile-time N)	left, right, stride, padded [cite: 5, 11]	Implicit via AccessorPolicy [cite: 5, 10]	Unstandardized
<code>std::mdarray</code> [cite: 8]	Owning Adapter	Static Rank (Compile-time N)	left, right, stride [cite: 8, 19]	Allocator-dependent ⁸	Unstandardized
Eigen Tensor	Owning / View	Static Rank (Compile-time N)	Row-Major / Column-Major	Host CPU & CUDA Device	Custom non-standard C++ API
PyTorch <code>at::Tensor</code>	Owning (Ref-counted)	Dynamic Rank (Runtime N)	Strided / Non-contiguous	Host CPU, CUDA, MPS, XLA	Native DLPack Protocol ²

Proposed std::tensor	Owning / Shared	Hybrid (Static N or Dynamic)	left, right, stride, padded	Explicit device_con text	Native DLManage dTensorVe rsioned [cite: 2, 4]
-------------------------	--------------------	---	-----------------------------------	--------------------------------	--

Technical Design Blueprint for std::tensor

The proposed std::tensor class template balances compile-time layout optimization with dynamic runtime rank capabilities. It acts as an owning multidimensional container while maintaining native interoperability with std::mdspan and std::linalg⁵.

Class Template Architecture and Signatures

Defined within the <tensor> header, std::tensor is structured as a specialization of std::basic_tensor:

```
C++
namespace std

// Special tag type designating dynamic runtime rank determination
struct dynamic_rank_t { explicit dynamic_rank_t() default
inline constexpr dynamic_rank_t dynamic_rank()

template
    typename Element Type
    typename ExtentsPolicy
    typename LayoutPolicy = std::layout_right
    typename Allocator = std::allocator<Element Type>
{
    class basic_tensor

// Type alias for statically ranked tensors (rank fixed at compile time)
template
    typename Element Type
    typename Extents
    typename LayoutPolicy = std::layout_right
    typename Allocator = std::allocator<Element Type>
{
    using tensor = basic_tensor<Element Type, Extents, LayoutPolicy, Allocator>
```

```
// Type alias for dynamically ranked tensors (rank determined at runtime)
template
    typename ElementType
    typename LayoutPolicy = std::layout_right
    typename Allocator = std::allocator<ElementType>
{
using dynamic_tensor = basic_tensor<ElementType, dynamic_rank_t, LayoutPolicy, Allocator>

// namespace std
```

Static vs. Dynamic Rank Mechanics

To support both fixed-rank mathematical operations and flexible runtime deep learning pipelines, ExtentsPolicy operates in two distinct modes:

1. **Static Rank Mode:** ExtentsPolicy is an instance of `std::extents<IndexType, Extents...>`⁵. The rank N is fixed at compile time (`Extents::rank()`), while individual dimension extents may be static (`std::static_extent`) or dynamic (`std::dynamic_extent`)⁵.
2. **Dynamic Rank Mode:** ExtentsPolicy is specified as `std::dynamic_rank_t`. The rank N is determined at construction time and stored in a lightweight runtime container (such as a stack-optimized `small_vector<size_t, 8>`). Striding and index calculation equations dynamically adapt to rank modifications without requiring template recompilation.

Striding Calculations and Memory Offset Equations

For a tensor of rank r with shape dimensions $\mathbf{E} = (E_0, E_1, \dots, E_{r-1})$, the offset calculation for an index tuple $\mathbf{i} = (i_0, i_1, \dots, i_{r-1})$ is governed by the active LayoutPolicy⁵:

- **Row-Major Layout (`std::layout_right`):**

$$S_{r-1} = 1, \quad S_d = \prod_{k=d+1}^{r-1} E_k \quad \text{for } 0 \leq d < r - 1$$

$$\text{Offset}(\mathbf{i}) = \sum_{d=0}^{r-1} i_d \times S_d$$

- **Column-Major Layout (`std::layout_left`):**

$$S_0 = 1, \quad S_d = \prod_{k=0}^{d-1} E_k \quad \text{for } 0 < d \leq r - 1$$

$$\text{Offset}(\mathbf{i}) = \sum_{d=0}^{r-1} i_d \times S_d$$

- **General Strided Layout (std::layout_stride):** Explicitly stores per-axis stride values $\mathbf{S} = (S_0, S_1, \dots, S_{r-1})$, allowing non-contiguous subviews, transposed aliases, and zero-stride broadcast projections¹³.

Storage Architecture and Reference-Counted Shared Memory

std::tensor manages memory allocations using a reference-counted storage handle (such as a polymorphic memory resource control block). This design provides several operational capabilities:

1. **Shallow Copy Slicing ($\mathcal{O}(1)$):** Slicing, reshaping, or transposing operations via submdspan return new std::tensor instances that share the underlying data buffer while maintaining independent index mapping metadata¹².
2. **Explicit Deep Copy Cloning:** Deep memory copying is isolated to explicit tensor::clone() operations, preventing accidental heavy allocations during function parameter passing.
3. **PMR Allocator Integration:** Support for std::pmr::memory_resource allows integration with custom allocation strategies, including arena allocators, memory pools, and host-pinned memory resources⁸.

Heterogeneous Execution Domain Awareness

Deep learning software executes across heterogeneous hardware domains, including CPU host memory, CUDA GPU allocations, ROCm device memory, and SYCL unified shared memory spaces². std::tensor incorporates explicit execution domain tracking into its storage model:

```
C++
```

```
namespace std
```

```
enum class device_type_t : int32_t
```

```

cpu = 1
cuda = 2
cuda_host = 3
opencl = 4
vulkan = 7
rocm = 10
rocm_host = 11
svcl = 12
}

struct device_context {
    device_type_t device_type; device_type_t cpu;
    int32_t device_id; 0
}

// namespace std

```

Each tensor instance stores a `device_context` descriptor alongside its memory handle. Attempting to directly dereference host pointers for GPU-resident tensors raises a runtime exception or triggers precondition violations under library hardening checks².

Interoperability Engine: Native C ABI Exchange via DLPack

To eliminate C++ tensor library fragmentation, `std::tensor` incorporates a native C ABI export and import engine based on the standard DLPack specification (`DLManagedTensorVersioned`)².

Standard C ABI Data Structures

The DLPack standard specifies C ABI data structures designed for zero-copy tensor exchange across frameworks and runtime boundaries²:

```

C
// Plain C Tensor Descriptor (non-owning layout ABI struct)
typedef struct {
    void *data;
    DLDevice_t device;
    int32_t ndim;
    DLDataType_t dtype;
    int64_t *shape;
    int64_t *strides;
}

```

```

uint64_t byte_offset
DLTensor

// Versioned Managed Tensor C ABI Structure for Zero-Copy Exchange
typedef struct DLManagedTensorVersioned {
    uint32_t version_major
    uint32_t version_minor
    DLTensor dl_tensor
    void manager_ctx
    void deleter(struct DLManagedTensorVersioned* self)
    DLManagedTensorVersioned

```

Zero-Copy Interoperability Protocols

std::tensor implements bidirectional, zero-copy conversion functions to export memory buffers to external C libraries or import buffers from runtimes such as PyTorch, TVM, and CPython¹:

1. Exporting std::tensor to DLManagedTensorVersioned

During export, std::tensor transfers ownership of its underlying storage handle to the manager_ctx pointer of a heap-allocated DLManagedTensorVersioned structure²:

```

C++
template <typename T, typename Extents, typename Layout, typename Allocator>
DLManagedTensorVersioned* to_dlpack(basic_tensor<T, Extents, Layout, Allocator>&& src) {
    auto managed = new DLManagedTensorVersioned();
    managed->version_major = 1 // DLPACK_MAJOR_VERSION
    managed->version_minor = 0

    // Allocate context block holding src's underlying storage reference
    auto ctx = new tensor_storage_handle(src.extract_storage());
    managed->manager_ctx = ctx

    managed->dl_tensor.data = ctx->data_ptr()
    managed->dl_tensor.byte_offset = src.byte_offset()
    managed->dl_tensor.device = src.device_context().to_dl_device()
    managed->dl_tensor.ndim = static_cast<int32_t>(src.rank())
    managed->dl_tensor.dtype = cxx_dtype_to_dlpack<T>()
    managed->dl_tensor.shape = ctx->shape_data() // Stored in context buffer
    managed->dl_tensor.strides = ctx->stride_data() // Stored in context buffer

```

```

        managed->deleter = [](DLManagedTensorVersioned* self) noexcept
        {
            if (!self) return;

            auto ctx = static_cast<tensor_storage_handle*>(self->manager.ctx);
            delete ctx; // Releases reference-counted memory handle
            delete self;
        };

        return managed;
    }

```

2. Importing DLManagedTensorVersioned into std::tensor

During import, std::tensor wraps the incoming DLManagedTensorVersioned structure. Its custom storage deleter invokes the source framework's C deleter function once all reference counts reach zero²:

```

C++
template<typename T>
dynamic_tensor<T> from_dlpack(DLManagedTensorVersioned* dlpack_tensor)
{
    if (!dlpack_tensor) throw std::invalid_argument("Null DLPack tensor pointer.");

    // Verify ABI version compatibility
    if (dlpack_tensor->version.major != 1)
    {
        if (dlpack_tensor->deleter) dlpack_tensor->deleter(dlpack_tensor);
        throw std::runtime_error("Incompatible DLPack ABI version.");
    }

    // Wrap external buffer with custom deleter calling DLPack deleter callback
    auto storage = make_dlpack_shared_storage(dlpack_tensor);
    return dynamic_tensor<T>(storage, dlpack_tensor->dl_tensor);
}

```

Property Mapping between std::tensor and DLPack ABI Fields

Proposed std::tensor Property	Corresponding DLPack C ABI Field	Mapping Mechanics and Conversion Rules
tensor::data()	DLTensor::data	Pointer to raw memory

		allocation without applying byte offset ²⁴ .
tensor::byte_offset()	DLTensor::byte_offset	Offset in bytes from allocation start to the first valid element ² .
tensor::rank()	DLTensor::ndim	Total rank count represented as a 32-bit signed integer ² .
tensor::shape()	DLTensor::shape	Pointer to array of int64_t values specifying dimensions along each axis ² .
tensor::strides()	DLTensor::strides	Pointer to element-based stride values per axis (never null in DLPack $\geq$) ² .
sizeof(T) & Type Category	DLDataType (code, bits, lanes)	Bitwise mapping to standard scalar codes (uint, int, float, bfloat) ²⁰ .
tensor::device()	DLDevice (device_type, device_id)	Enum conversion matching target hardware execution contexts ² .
Memory Lifetime Management	deleter / manager_ctx	Custom deleter invocation ensuring safe cross-framework deallocation ² .

Mathematical and Algorithmic Integration

To support high-performance numeric processing, std::tensor integrates with existing C++ numeric algorithms, vectorization utilities, and dense linear algebra interfaces⁶.

Interoperability with std::mdspan and std::linalg

std::tensor provides explicit view extraction functions to generate non-owning std::mdspan instances⁵:

```

C++
std::tensor<float, std::extents<size_t, 128, 64>> A = tensor<128, 64>()

// Generate a non-owning std::mdspan view over tensor memory
std::mdspan A_view = A.tensor_to_mdspan()

// Pass views directly into standard dense linear algebra routines
std::tensor<float, std::extents<size_t, 64, 32>> B = tensor<64, 32>()
std::tensor<float, std::extents<size_t, 128, 32>> C = tensor<128, 32>()

std::linalg::matrix_product(
    std::execution::par_unseq,
    A.tensor_to_mdspan(),
    B.tensor_to_mdspan(),
    C.tensor_to_mdspan(),
)

```

This design allows `std::tensor` to serve as an owning storage backend for `std::linalg` routines⁶.

Multi-Dimensional Indexing and Slicing

`std::tensor` supports variadic multi-parameter operator`[]` syntax (standardized in C++23) for element access⁵:

```

C++
std::dynamic_tensor<double> t({3, 4, 5})

// Direct variadic element access
t[1][2][3] = 3.14159

// Integrated submdspan subview creation
auto slice_view = std::submdspan(
    t.to_mdspan(),
    1,
    std::full_extent,
    std::strided_slice{offset = 0, extent = 5, stride = 2}
)

```

Broadcasting Rules and Array Arithmetic

`std::tensor` implements standard multidimensional broadcasting rules for binary element-wise operations. When evaluating $C = A + B$ for tensors of shape $(3, 1, 5)$ and $(4, 5)$, the indexing engine computes an output domain shape of $(3, 4, 5)$ using broadcast stride rules²⁰:

$$\text{stride}_d^{A'} = \begin{cases} \text{stride}_d^A & \text{if } \text{extent}_d^A = \text{extent}_d^C \\ 0 & \text{if } \text{extent}_d^A = 1 \end{cases}$$

Setting the stride along axis d to zero broadcasts scalar values across that dimension without memory duplication. Element dereferencing maps directly to `std::simd` vector registers, maximizing vector execution throughput¹⁸.

Technical Considerations and ISO Committee Trade-offs

Designing `std::tensor` requires addressing specific technical challenges identified during standard committee reviews of `std::mdspan` and `std::mdarray`⁵.

Resolving LEWG Container Adapter Feedback (P1684 / P3308)

Reviews by the Library Evolution Working Group (LEWG) raised questions regarding constructor overloading and moved-from object states in owning multidimensional containers⁸. `std::tensor` addresses these points through specific design mechanisms:

1. **Moved-From State Guarantees:** Moving a `std::tensor` clears its shape descriptor and sets its internal storage handle to `nullptr`. Post-move operations, except destruction and assignment (operator=), trigger precondition violations enforced under standard hardening guidelines (P3471)⁸.
2. **Disambiguation via `in_place_t` Constructors:** To prevent constructor ambiguity between flat storage initializers and multidimensional shape parameters, `std::tensor` provides `std::in_place_t` constructors⁸:

```
C++
// Constructs flat element storage in place using forwarded arguments
std::tensor<float, std::extent<size_t, 4, 4>>> t;
std::in_place(t, 1.0f, 2.0f, 3.0f /*...*/ 16.0f);
```

3. **Initializer List Deduction Guides:** Deduction rules support direct construction from nested initializer lists for static rank tensors⁸:

```
C++
// Class Template Argument Deduction infers tensor<int, extents<size_t, 2, 3>>
std::tensor A = {{ 1 2 3 } { 4 5 6 }}
```

Static Template Specialization vs. Runtime Type Erasure

Deep learning engines require type-erased runtime tensors (at::Tensor), whereas high-performance C++ code relies on compile-time static typing (std::tensor<T, Extents>). To support both paradigms, std::tensor uses static layout engines by default. For runtime execution pipelines, a type-erased container class, std::any_tensor, wraps an underlying std::basic_tensor instance and provides runtime dynamic dispatch:

```
C++
namespace std {

class any_tensor {
    std::shared_ptr<void> storage_ptr;
    DLDataType dtype;
    device_context device;

public:
    template<typename T, typename Extents, typename Layout>
    any_tensor(basic_tensor<T, Extents, Layout>& t) {
        // ...
    }

    DLDataType dtype() const noexcept { return dtype; }

    template<typename T>
    basic_tensor<T, dynamic_rank_t>& cast() {
        if (cxx_dtype_to_dlpack<T>() != dtype) throw std::bad_any_cast();
        return static_cast<basic_tensor<T, dynamic_rank_t>>(*storage_ptr.get());
    }
};

// namespace std
```

Memory Alignment and SIMD Vectorization

Vectorized code generation requires aligned memory access guarantees¹⁰. std::tensor ensures alignment by integrating overaligned allocations with padded memory layouts (layout_left_padded and layout_right_padded via P2642)¹¹.

This guarantees that the leading dimension stride (LDA) aligns with hardware vector register boundaries (such as 64-byte alignment for AVX-512 or ARM SVE), avoiding performance penalties from unaligned memory loads¹⁰.

ISO Standardization Roadmap

To progress `std::tensor` through the ISO C++ standardization process for target inclusion in C++29, proposal work is structured into three execution phases:

Standardization Phase	Technical Scope & Deliverables	WG21 Working Group Focus
Phase 1: Core Tensor Mechanics	- Standardize <code>std::basic_tensor</code>, <code>std::tensor</code>, and <code>std::dynamic_tensor</code>. - Implement variadic <code>operator[]</code> indexing and CTAD deduction guides⁵. - Integrate <code>std::mdspan</code> view extraction and <code>submdspan</code> slicing support⁵.	LEWG (Library Evolution) & LWG (Library Working Group)
Phase 2: Heterogeneous Support	- Integrate <code>std::pmr::memory_resource</code> support⁸. - Formally define <code>device_context</code> and hardware memory domain semantics. - Extend parallel algorithms (<code>std::execution::par_unseq</code>) to support asynchronous tensor operations¹⁰.	SG14 (Low Latency) & SG19 (Machine Learning)
Phase 3: C ABI DLPack	- Standardize native <code><dlpack.h></code> header bindings.	LEWG & International

Bridge	- Implement to_dlpack and from_dlpack zero-copy conversion routines². - Verify cross-language interop across PyTorch, TVM, TensorFlow, and OpenCV¹.	Standardization Committee
---------------	--	---------------------------

Architectural Synthesis

The proposed `std::tensor` library resolves data structure fragmentation across C++ numeric computing libraries. By combining the zero-overhead abstraction model of `std::mdspan`⁵ with dynamic runtime rank flexibility, heterogeneous memory awareness, and native C ABI exchange via DLPack², `std::tensor` provides a foundational, owning multidimensional container for the ISO C++ Standard Library. This standardized abstraction enables seamless data exchange across machine learning libraries, scientific computing packages, and cross-language runtime environments while preserving C++ compile-time performance guarantees.

Works cited

1. Creating an Application - NVIDIA Docs, https://docs.nvidia.com/holoscan/archive/0.5.1/holoscan_create_app.html
2. C API (dlpack.h) - DMLC, https://dmlc.github.io/dlpack/latest/c_api.html
3. [RFC] Adopt DLPack as cross-language C ABI stable data structure for array exchange #1, <https://github.com/data-apis/consortium-feedback/issues/1>
4. Tensor and DLPack — tvml-ffi, <https://tvm.apache.org/ffi/concepts/tensor.html>
5. MDSPAN - Open-std.org, <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2022/p0009r18.html>
6. What new feature would you like to see in C++26? : r/cpp - Reddit, https://www.reddit.com/r/cpp/comments/1bqv7w8/what_new_feature_would_you_like_to_see_in_c26/
7. A free function linear algebra interface based on the BLAS - Open-std.org, <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/p1673r13.html>
8. mdarray design questions and answers - Open-std.org, <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2024/p3308r0.html>
9. WG21, aka C++ Standard Committee, May 2024 Mailing (pre-St. Louis) : r/cpp - Reddit, https://www.reddit.com/r/cpp/comments/1cy8k8o/wg21_aka_c_standard_committee_may_2024_mailing/
10. MDSPAN - A Deep Dive Spanning C++, Kokkos & SYCL, <https://nwcpp.org/talks/2023/MDSPAN.pdf>

11. `cpp-proposals-pub/layout_padded/layout_padded.bs` at master · ORNL/cpp-proposals-pub - GitHub, https://github.com/ORNLCPP/cpp-proposals-pub/blob/master/layout_padded/layout_padded.bs
12. Initialise 1D vector using 2D vector - Programming - Arduino Forum, <https://forum.arduino.cc/t/initialise-1d-vector-using-2d-vector/1004096>
13. Future-proof `submdspan_mapping` - Open-std.org, <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/p3663r1.html>
14. Fix `submdspan` for C++26 - Open-std.org, <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2024/p3355r0.html>
15. [DOC] `mdspan/mdarray` quick-start tutorial · Issue #654 · NVIDIA/raft, <https://github.com/NVIDIA/raft/issues/654>
16. `span` for projections? : r/cpp - Reddit, https://www.reddit.com/r/cpp/comments/18xgviv/span_for_projections/
17. Future-proof `submdspan_mapping` - Open-Std.org, <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/p3663r3.html>
18. C++26 - ISciNumPy.dev, <https://iscinumpy.dev/post/cpp-26/>
19. `mdarray`: An Owning Multidimensional Array Analog of `mdspan` - Open-std.org, <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/p1684r5.html>
20. Tensors — `augpy` documentation, <https://augpy.readthedocs.io/en/latest/cpp/tensor.html>
21. ISO C++ std-proposals List: Re: [std-proposals] Interest in Linear, <https://lists.isocpp.org/std-proposals/2023/04/6371.php>
22. `pytorch` | Terra Incognita, <https://blog.christianperone.com/tag/pytorch/>
23. Report from the Croydon 2026 ISO C++ Committee meeting - `mp-units`, <https://mpusz.github.io/mp-units/HEAD/blog/2026/03/28/report-from-the-croydon-2026-iso-c-committee-meeting/>
24. C++ API Reference (Extras) - `nanobind` documentation, https://nanobind.readthedocs.io/en/latest/api_extra.html
25. Cologne 2019 LEWG Summary - Open-std.org, <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/n4823.pdf>
26. P1684 R5 `mdarray`: An Owning Multidimensional Array Analog of `mdspan` · Issue #461 · `cplusplus/papers` - GitHub, <https://github.com/cplusplus/papers/issues/461>
27. 2023-11 Kona ISO C++ Committee Trip Report — Second C++26 meeting! : r/cpp - Reddit, https://www.reddit.com/r/cpp/comments/17vnfqq/202311_kona_iso_c_committee_trip_report_second/